

First Steps — Arinze

Your lane: the entire minigame system — the host apps, all ten games, and the wiring that puts them in the story. **Cadence:** one or two games a week, each taken all the way to playable-in-story before starting the next. **You block:** everyone, once — step 1. After that, nobody waits on you and you wait on nobody.

You own a full vertical slice. No handoffs, no integration meetings, no waiting for someone else's shell. The tradeoff is that the host system is now yours too — but it's smaller than it sounds, because `Terminal.jsx` already does the whole job for the Ch 5 decrypt. You're generalising working code, not writing new architecture.

1. Commit the working tree · 10 min — DO THIS FIRST

~50 modified files are uncommitted, including the entire Season One integration. Nobody can branch cleanly until this lands.

```
git status
git add -A
git commit -m "Season One complete: Ch 8-12 integrated, endgame mechanics, docs"
git push
git branch feat/minigames && git branch feat/ui && git branch feat/ch12
git push --all
```

Done when: `git status` is clean, branches pushed. **Unblocks:** Luca and Ben.

2. Play the full season · half a day

One route, start to finish, no skipping. Notes on:

- Where the pacing sags — those are your real minigame slots, not the ones proposed on paper
- Where a proposed minigame would *interrupt* rather than deepen
- Anything broken or confusing (hand that list to Luca — it's his lane now)

Why before building: you're about to spend months on ten games. A day of play tells you which ones the game actually wants.

3. Generalise the host · half a day

This is the piece that used to be Luca's. It's small.

`src/screens/Terminal.jsx` already implements the entire host contract for the Ch 5 decrypt: it reads `flags.__minigame__`, runs the countdown, auto-fails on timeout, shows a result, writes flags, and calls `engine.advanceBeat()`. It's just hardcoded to one game.

Pull that logic into a reusable host and point three thin apps at it:

```
src/screens/minigames/
  MinigameHost.jsx  ← the extracted logic, mounts a game by id
  registry.js       ← { ch6_marker_watch: { app:'lens', component: MarkerWatch } }
  games/           ← one file per game
```

Then Terminal (extend), Lens (new), Feeds (new) each render `<MinigameHost app="..." />` plus an idle screen.

Done when: the Ch 5 decrypt still works, running through the new host instead of hardcoded.

Why first: it makes step 5 possible. Do it before you have games to migrate, not after.

4. Build Marker Watch (Ch 6) · your call

Deliberately the hardest one first — it's furthest from the terminal grid, so it stress-tests your host design while it's still cheap to change.

From Ben's scene: Vi says *"there's no one here to watch them for me."* You plot the markers from Loray's descriptions, something pulls your attention away, and you mark which ones moved.

Signature stays fixed (docs/MINIGAME_CONTRACT.md):

```
export default function MarkerWatch({ config, timeLeft, onResolve }) { ... }
```

No engine imports, no beat advancing, no flag writing, no timer of your own — the host does all of that. End with `onResolve({ outcome: 'success' | 'failure' })`.

5. Wire it — the first full slice · half a day

This is the new part, and the reason your cadence is one to two games a week rather than five.

1. **Converter** — extend [MINIGAME] in `scripts/convert-story.js` to carry a game id, an app, and an optional quality flag (docs/MINIGAME_CONTRACT.md §4). One regex, same shape as the existing one.
2. **Master file** — add the [MINIGAME] block and its success/failure beats to `CH6_MASTER.md`.
3. **Seam lines** — draft them yourself in Ben's voice while you're in there. Don't wait on him; he reviews in batches later.
4. **Convert and validate:**

```
node scripts/convert-story.js docs/chapters/CH6_MASTER.md --out /tmp/ch6.json  
npm run validate
```

5. **Play the chapter** and confirm both outcomes route correctly.

Done when: you can play Ch 6 from the start of the chapter, hit the minigame in the Lens app, and land on the right beat whether you succeed or fail.

That's one complete slice. Every game after this is the same five steps, and they get faster.

After that

Terminal family as a batch — Ch 2, Ch 8, Ch 9 ×2 — they share the grid lineage so they go quickest. Then Ch 7 Verify, then Ch 3 and Ch 12's Drone Work once the Feeds shell exists.

Send Ben your draft seam lines in batches of two or three. He rewrites them in his own voice; you drop the replacements in. No blocking either direction.